Capítulo 4. Capa de Servicio

La **capa de servicio** es una pieza clave en el diseño de aplicaciones backend. Actúa como un intermediario entre los controladores (que manejan las solicitudes HTTP) y los repositorios (que interactúan con la base de datos). Su objetivo principal es encapsular la lógica de negocio, lo que permite separar responsabilidades y mejorar la organización del código.

4.1. La Capa de Servicio

La capa de servicio es responsable de:

1. **Implementar la Lógica de Negocio:**
   - Contiene las reglas y procesos específicos de la aplicación.
   - Ejemplo: Validar datos, calcular valores, realizar operaciones complejas.
2. **Coordinar Acciones entre Componentes:**
   - Interactúa con los repositorios para acceder a los datos.
   - Procesa los datos antes de enviarlos al controlador.
3. **Mejorar la Modularidad:**
   - Permite reutilizar la lógica de negocio en diferentes partes de la aplicación.
   - Facilita las pruebas unitarias al aislar la lógica de negocio.

Ejemplo de una Clase de Servicio:

```
@Service
public class SaludoService {

    public String generarSaludo(String nombre) {
        if (nombre == null || nombre.isEmpty()) {
            throw new IllegalArgumentException("El nombre no puede estar vacío");
        }
        return "¡Hola, " + nombre + "!";
    }
}
```

- **@Service:** Indica que esta clase es un componente de la capa de servicio.
- **Lógica de Negocio:** Valida el nombre y genera un saludo personalizado.

4.2. Gestión de Errores

La gestión de errores es fundamental en cualquier aplicación. En la capa de servicio, puedes manejar errores específicos y lanzar excepciones personalizadas para que los controladores las gestionen adecuadamente.

Excepciones Personalizadas:

```
public class SaludoException extends RuntimeException {
    public SaludoException(String mensaje) {
        super(mensaje);
    }
}
```

Manejo de Errores en el Servicio:

```java
@Service
public class SaludoService {

    public String generarSaludo(String nombre) {
        if (nombre == null || nombre.isEmpty()) {
            throw new SaludoException("El nombre no puede estar vacío");
        }
        return "¡Hola, " + nombre + "!";
    }
}
```

- **Controlador que Maneja la Excepción:**

```java
@RestController
@RequestMapping("/api")
public class SaludoController {

    @Autowired
    private SaludoService saludoService;

    @GetMapping("/saludo/{nombre}")
    public ResponseEntity<String> saludar(@PathVariable String nombre) {
        try {
            String mensaje = saludoService.generarSaludo(nombre);
            return ResponseEntity.ok(mensaje);
        } catch (SaludoException e) {
            return ResponseEntity.badRequest().body(e.getMessage());
        }
    }
}
```

- **ResponseEntity:** Permite devolver respuestas HTTP con códigos de estado específicos.

4.3. Capa de Servicio mediante Interfaces y Clases

Usar interfaces en la capa de servicio mejora la modularidad y facilita las pruebas unitarias. Las interfaces definen los métodos que deben implementarse, mientras que las clases concretas proporcionan la implementación.

Ejemplo de Interfaz de Servicio:

```java
public interface SaludoService {
    String generarSaludo(String nombre);
}
```

Implementación de la Interfaz:

```java
@Service
public class SaludoServiceImpl implements SaludoService {

    @Override
    public String generarSaludo(String nombre) {
        if (nombre == null || nombre.isEmpty()) {
            throw new IllegalArgumentException("El nombre no puede estar vacío");
        }
        return "¡Hola, " + nombre + "!";
```

```
    }
}
```

- **Ventajas:**
  - Facilita el uso de inyección de dependencias.
  - Permite intercambiar implementaciones sin modificar el controlador.

4.4. CommandLineRunner

Spring Boot incluye la interfaz `CommandLineRunner`, que permite ejecutar código automáticamente cuando la aplicación arranca. Esto es útil para inicializar datos o realizar tareas de configuración.

Ejemplo de `CommandLineRunner`:

```java
@Component
public class InicializadorDatos implements CommandLineRunner {

    @Autowired
    private SaludoService saludoService;

    @Override
    public void run(String... args) throws Exception {
        System.out.println(saludoService.generarSaludo("Mundo"));
    }
}
```

- **`CommandLineRunner`:** Define un método `run` que se ejecuta al iniciar la aplicación.
- **Uso Práctico:** Inicializar datos en la base de datos, registrar logs, etc.

Ejercicios de Ampliación

1. **Crear un Servicio para Calcular el Área de un Círculo:**
   - Implementa una clase de servicio que calcule el área de un círculo dado su radio.
   - Maneja errores si el radio es negativo o inválido.
2. **Gestión de Excepciones Personalizadas:**
   - Crea una excepción personalizada para manejar errores en un servicio que valida datos de usuario.
3. **Usar `CommandLineRunner` para Inicializar Datos:**
   - Configura un `CommandLineRunner` que inicialice una lista de usuarios en memoria al arrancar la aplicación.

Conclusión

La **capa de servicio** es esencial para encapsular la lógica de negocio y mejorar la organización del código. Al usar interfaces, gestionar errores y aprovechar herramientas como `CommandLineRunner`, puedes crear aplicaciones backend más robustas y mantenibles.